The first argument to BigDecimal is the value you are converting, and the second argument is the number of digits of precision you want. Depending on what the first argument is to BigDecimal, you may or may not be required to specify the second argument.

The number of digits of precision is the number of digits used regardless of what side of the decimal point the digit is on. In these examples, you can see that the number of digits of precision changes the value of the BigDecimal as it is converted to a float using to_f:

```
require "bigdecimal"
BigDecimal(31.419, 5).to_f # => 31.419
BigDecimal(31.419, 4).to_f # => 31.42
BigDecimal(31.419, 3).to_f # => 31.4
BigDecimal(31.419, 2).to_f # => 31.0
BigDecimal(31.419, 1).to_f # => 30.0
```

If the first argument is a Complex, an Integer, a String, or another BigDecimal, the conversion does not need to know the digits of precision. If the first argument is a Float or a Rational, then you must include the digits of precision.

Strings are parsed and assumed to use the number of digits equivalent to characters in the string:

```
require "bigdecimal"
BigDecimal("31.419").to_f # => 31.419
```

Any other type as the first value, and Ruby attempts to convert it to a string using to_str and parses that string (per the documentation, it does not appear that Ruby attempts to covert to a number). If the conversion fails, the method returns nil. An optional keyword argument, exception: true, will cause BigDecimal to raise an exception in that case instead.

Once you have a BigDecimal, you can basically use it like any other numeric. If it's the left side of an operation, the result will also be a BigDecimal:

```
require "bigdecimal"
x = BigDecimal("31.419") + 5.3
x.class   # => BigDecimal
```

You can convert out of BigDecimal with the usual suspects, to_i, to_f, to_s, and to_r for Rational. There's also a module called bigdecimal/util—requiring that gives you a to_d method on Integer, Float, Rational, and String that converts to a BigDecimal.

A few special values—Infinity, +Infinity, -Infinity, and NaN—are all valid values, which you normally trigger by dividing by zero:

```
require "bigdecimal"
BigDecimal("1.0") / BigDecimal("0.0") # => Infinity
BigDecimal("0.0") / BigDecimal("0.0") # => NaN
```

BigDecimal also allows you to control how values are rounded to the number of significant digits. If you are like us, you were taught in school to round to the closest digit, and that the value in the middle—5—rounds up. It turns out that is just a convention, and other conventions are also valuable. You can change the rounding mode globally with BigDecimal.mode(BigDecimal::ROUND_MODE, [value]), where the value may be:

- BigDecimal:ROUND_CEILING—Always round toward the higher number.
- BigDecimal:ROUND_DOWN—Always round toward zero, so 8.2 rounds to 8.